Chapter 5. Scoped Values

> *He who does not place things in their proper place has committed injustice.*
>
> —Ali ibn Abi Talib (may God be pleased with him)

In this chapter, we will explore `ScopedValue`, a powerful addition to Java that was finalized in JDK 25. It provides a structured way to bind values to a specific scope while remaining accessible and consistent with the context. Unlike traditional `ThreadLocal` variables, which can be cumbersome and prone to memory leaks, this offers a cleaner and more efficient approach when dealing with multithreaded applications. In this chapter, we will examine why `ScopedValue` is a better replacement for `ThreadLocal` and its API and then walk through practical use cases.

Let's begin.

The Burden of Passing Context

There are often scenarios in which we need to share data among various parts of the code where simply using method arguments isn't feasible. This is especially common when our application depends on a framework. To illustrate the idea, let's consider an example.

Imagine a job-scheduling framework in which user code registers tasks that the framework will execute. Whenever the framework runs a job, it creates a `JobContext` object containing metadata such as the job's name, priority, and scheduling constraints. This context is essential for framework operations but is largely irrelevant to user code.

The following example demonstrates a typical job-scheduling framework that suffers from what we call the "parameter passing problem":

```
import java.time.Instant;
import java.util.HashMap;
import java.util.Map;

interface Job {
    void execute(JobContext context);
}
```

```
enum Priority { LOW, MEDIUM, HIGH }

public record JobContext(String jobName, Priority priority,
                         Map<String, Object> metadata) {
    public JobContext(String jobName, Priority priority) {
        this(jobName, priority, new HashMap<>());
        metadata.put("jobName", jobName);
        metadata.put("priority", priority);
        metadata.put("creationTime", Instant.now());
    }

    public Object getMetadataValue(String key) {
        return metadata.get(key);
    }
}
```

The framework's core scheduling logic creates and manages the job context:

```
// Framework code
public class JobScheduler {
    public void schedule(Job job, String jobName, Priority priority) { ❶
        JobContext context = new JobContext(jobName, priority);
        runJob(job, context);
    }

    private void runJob(Job job, JobContext context) { ❷
        // The framework calls user code here, passing the context
        job.execute(context);
    }

    public Object getJobMetadata(String key, JobContext context) { ❸
        if (context == null) {
            return null;
        }
        return context.getMetadataValue(key);
    }
}
```

To use this framework, we must implement the `Job` interface and work with the framework's `context` object:

```
public class UserJob implements Job {
    private final JobScheduler jobScheduler;

    public UserJob(JobScheduler jobScheduler) {
        this.jobScheduler = jobScheduler;
    }
```

```
    @Override
    public void execute(JobContext context) { ❹
        System.out.println("User job is running!");

        // User code calls back into the framework to retrieve metadata
        Object creationTime
            = jobScheduler.getJobMetadata("creationTime", context); ❺
        System.out.println("Job creation time: " + creationTime);

        // Any helper methods also need the context parameter
        processJobData(context); ❻
    }

    private void processJobData(JobContext context) { ❼
        // Even though this method might not directly use context,
        // it needs the parameter to pass to other framework methods
        Object priority = jobScheduler.getJobMetadata("priority", context)
        System.out.println("Processing job with priority: " + priority);
    }
}
```

❶ The framework creates a `JobContext` containing metadata needed throughout the job's lifecycle.

❷ The `context` must be passed down to user code, even though users shouldn't need to understand framework internals.

❸ Framework methods require the `context` parameter to access metadata, forcing it back up the call chain.

❹ User implementations must accept the `JobContext` parameter in their `execute()` method.

❺ When user code needs framework services, it must pass the `context` back to the framework.

❻ Helper methods in user code become contaminated with framework parameters.

❼ Every method in the user's call chain needs the `context` parameter, even if it doesn't directly use it.

While this approach functions correctly, it introduces several architectural problems that become more pronounced as applications scale.

Parameter Pollution

The `JobContext` is primarily a framework construct that developers using the framework shouldn't need to understand. However, because the framework must manage its internal context across the call chain, from `schedule()` down into user code in `execute()`, and then back into the framework inside `getJobMetadata()`, user code is forced to carry around `JobContext` parameters.

Every method that participates in the job execution flow must declare this parameter, even if the method itself doesn't use any context information. This pollutes method signatures with framework-specific details that have no business meaning to the application logic.

Interface Brittleness

Consider what happens when the framework evolves. If you later expand `JobContext` with additional data, say, a new `job category` field, a distributed tracing context, or a logging reference, you might need to modify every method signature in user code that passes context around. While the `JobContext` class itself maintains backward compatibility, the requirement to thread it through user code means that any expansion of the framework's needs ripples through the entire user codebase.

Coupling and Testability

User code becomes tightly coupled to framework implementation details. Testing individual methods becomes more complex because you must always provide a valid `JobContext`, even for tests that focus on business logic unrelated to the framework.

This coupling also makes it more difficult to migrate between different frameworks or to extract business logic for reuse in various contexts.

Introducing ThreadLocal

To circumvent this problem, the framework code can be designed intelligently using `ThreadLocal`, a tool commonly used in such code.

Let's reimplement the preceding code using `ThreadLocal`:

```
public class JobScheduler {
    private static final ThreadLocal<JobContext> jobContextHolder =
        new ThreadLocal<>(); ❶

    public void schedule(Job job, String jobName, Priority priority) {
        JobContext context = new JobContext(jobName, priority);
        try {
            jobContextHolder.set(context); ❷
            runJob(job);
        } finally {
            jobContextHolder.remove(); ❸
        }
    }

    private void runJob(Job job) { ❹
```

```
            job.execute();
        }

        public Object getJobMetadata(String key) {
            JobContext context = jobContextHolder.get(); ❺
            return (context != null) ? context.getMetadataValue(key) : null;
        }
    }
```

❶ Creates a `ThreadLocal` variable to hold the `JobContext` for each thread.

❷ Sets the `context` in the current thread before executing the job.

❸ Always remove the `context` in a `finally` block to prevent memory leaks.

❹ The `runJob` method no longer needs context parameters.

❺ Framework methods can retrieve context from `ThreadLocal` whenever needed.

Now the user code becomes much cleaner, with no need to handle framework context:

```
public class UserJob implements Job {
    private final JobScheduler jobScheduler;

    public UserJob(JobScheduler jobScheduler) {
        this.jobScheduler = jobScheduler;
    }

    @Override
    public void execute() { ❶
        System.out.println("User job is running!");
        // No context parameter needed - framework handles it internally
        Object creationTime = jobScheduler.getJobMetadata("creationTime");
        System.out.println("Job creation time: " + creationTime);
        // Helper methods are now clean
        processJobData();
    }

    private void processJobData() { ❸
        // Clean method signature - no framework parameters
        Object priority = jobScheduler.getJobMetadata("priority");
        System.out.println("Processing job with priority: " + priority);
    }
}
```

❶ User code is completely freed from framework context concerns.

❷ Framework services are accessible without any context parameters.

❸ Helper methods have clean signatures focused on business logic.

This approach solves the parameter-passing problem effectively. User code no longer needs framework-specific parameters; we can now focus on our own business logic. The framework's internal context management is now completely hidden from user code.

This pattern is so effective that virtually all modern frameworks use some form of `ThreadLocal` for context management. Spring Framework, for example, uses `ThreadLocal` extensively for security contexts, transaction contexts, and request contexts.

Limitations of ThreadLocal Variables

Although having `ThreadLocal` in code seems intelligent and useful, `ThreadLocal` variables have many inherent design flaws. Let's discuss them.

First, `ThreadLocal` offers unconstrained mutability. Any code that can call the `get()` method of a `ThreadLocal` variable can also call `set()`, allowing data to change at any time. This makes it difficult to track when and where data is modified.

Consider the following code:

```
public class MutableLoggingContext {
 // A ThreadLocal holding the current log level
 private static final ThreadLocal<String> LOG_LEVEL = new ThreadLocal<>();

 public static void setLogLevel(String level) {
   LOG_LEVEL.set(level); ❷
 }

 public static String getLogLevel() {
   return LOG_LEVEL.get();
 }

 public static void log(String message) {
   System.out.println("[" + getLogLevel() + "] " + message);
 }

 public static void main(String[] args) throws InterruptedException {
   setLogLevel("INFO");
   log("Starting process..."); ❸
   Thread thread = new Thread(() -> {
     setLogLevel("DEBUG"); ❹
     log("Thread-specific debug mode enabled");
   });
   thread.start();
```

```
        Thread.sleep(100); // Give the other thread time to run
        log("Main thread still at INFO level"); ❺
    }
}
```

In this example:

❶ Any code can access this static `ThreadLocal`.

❷ The log level can be changed from anywhere in the codebase.

❸ The main thread sets its log level to `INFO`.

❹ The child thread independently sets its own log level to `DEBUG`.

❺ The main thread's log level remains `INFO` (thread-local isolation).

As you can see, setting the log level can be done anywhere, leading to confusion about where it was set.

Second, it offers an unbounded lifetime. Once a thread-local variable is set, it remains for the life of that thread unless explicitly removed. This may not be a problem in regular threads, but we now use a thread pool where the same threads are reused repeatedly. This can lead to a data leak from one task to another if `remove()` is forgotten. It can cause memory leaks or even security vulnerabilities if sensitive data persists unintentionally.

Let's examine the following example:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadLocalLeakExample {
  private static final ThreadLocal<String> currentUser = new ThreadLocal<>

  public static void main(String[] args) throws InterruptedException {
    try (ExecutorService executor = Executors.newFixedThreadPool(1)) {❶
      // The first task sets the current user
      executor.submit(() -> {
        currentUser.set("Alice"); ❷
        System.out.println("Task 1: currentUser = " + currentUser.get());
        // Forgot to call currentUser.remove()! ❸
      });
      Thread.sleep(100); // Ensure task 1 completes
      // The second task reuses the same thread
      executor.submit(() -> {
        System.out.println("Task 2: Leaked value = " + currentUser.get());
        currentUser.set("Bob");
        System.out.println("Task 2: currentUser = " + currentUser.get());
        currentUser.remove(); //
      });
```

```
    }
  }
}
```

This example demonstrates the following:

1. Single-thread pool ensures the same thread handles both tasks.
2. First task sets a thread-local value.
3. Missing cleanup allows the value to persist.
4. Second task sees the leaked value from the first task.

This can cause memory leaks when objects remain referenced longer than intended, security vulnerabilities when sensitive data like authentication tokens or user contexts leaks between unrelated requests, and incorrect behavior when tasks operate with stale or incorrect context.

When using `InheritableThreadLocal`, child threads automatically inherit values from their parent thread. While this can be convenient, it becomes expensive when creating many child threads:

Now let's look at the next example:

```
public class InheritanceOverheadExample {
  private static final InheritableThreadLocal<byte[]> LARGE_DATA =
      new InheritableThreadLocal<>(); ❶

  public static void main(String[] args) {
    // Parent thread sets a large object
    LARGE_DATA.set(new byte[10_000_000]); // 10MB ❷
    // Create multiple child threads
    for (int i = 0; i < 100; i++) {
      new Thread(() -> {
        // Each child thread gets a reference to the parent's data ❸
        byte[] inherited = LARGE_DATA.get();
        System.out.println("Child has access to " +
            inherited.length + " bytes");
      }).start();
    }
  }
}
```

Problems with inheritance:

1. `InheritableThreadLocal` automatically copies values to child threads.
2. Large objects are referenced by all child threads.
3. Even if children never modify the data, they maintain references.

This example clearly demonstrates the downside of using `InheritableThreadLocal` when large objects are stored. Even though the child threads do not explicitly set or alter the local variable, they automatically inherit their own separate copy of the parent's data, resulting in duplicate memory usage.

Now that we understand the problem with the `ThreadLocal` variable, what would be the solution?

Toward Lightweight Sharing

The limitations of `ThreadLocal` become even more apparent with the introduction of virtual threads (JEP 444). Unlike traditional platform threads, where each thread has its own dedicated OS resource, virtual threads allow a single OS thread to host thousands or even millions of lightweight threads. While it's technically possible for each virtual thread to maintain its own `ThreadLocal` data, the memory overhead quickly becomes unsustainable. Imagine a million virtual threads, each carrying its own chunk of state—the cost would be staggering. Clearly, we need a better approach.

Since virtual threads are short-lived, the problem of long-lived thread locals is less dire, as garbage collection will remove them; still, the memory overhead of so many duplicates remains. Ideally, we want a new mechanism that allows us to store inheritable, per-thread data without incurring numerous copies. If the data is immutable, so much the better: one shared version can be referenced by child threads without additional duplication. We also need a bounded lifetime for this data; once the method sharing the data has wrapped up, any attached thread locals should lose their relevance. After all, they're meant to be a convenient place to hold state for the duration of a task, not a perpetual stash of memory.

That's where the new API `ScopedValue` comes into the picture. Let's break it down.

Core Components of ScopedValue

A `ScopedValue` acts as an implicit method parameter, allowing data to be passed through a sequence of method calls without explicitly declaring it in each method's signature. This facilitates cleaner and more maintainable code, especially when dealing with deeply nested method calls or callback structures.

It has three main characteristics:

Immutability

Once a `ScopedValue` is bound to a value within a specific scope, it cannot be altered, ensuring consistent and predictable behavior throughout its lifespan.

Thread-scoped binding

Bindings are confined to the current thread, preventing unintended data sharing across threads and enhancing thread safety.

Bounded lifetime

The binding of a `ScopedValue` is limited to the duration of a specific code block, after which it becomes unbound, aiding in resource management and reducing the risk of memory leaks.

NOTE

The `ScopedValue`, explored in this chapter, is available since JDK 25. If you are using an older JDK, you can enable it using the `--enable-preview` flag during compilation and runtime. To compile and run your code in JDK 24, for example, use the following commands:

- With `javac`: `javac --release 24 --enable-preview Main.java`
- With `java`: `java --enable-preview Main`
- With the source code launcher: `java --enable-preview Main.java`
- With JShell: `jshell --enable-preview`

We anticipate this feature will be integrated into future JDK releases without requiring any modifications to your code.

To use a scoped value, we first declare it as a static final field. We declare `ScopedValue` the same way we declare `ThreadLocal`:

```
private static final ScopedValue<String> NAME = ScopedValue.newInstance();
```

We don't use the `new` operator to instantiate; rather, we use the factory method to instantiate because its constructors are deliberately set to private.

Then, we can bind a value to the scoped value and execute code within that scope. This is achieved using the `where()` and `run()` methods:

```
ScopedValue.where(NAME, "duke").run(() -> doSomething());
```

In this example, the `where()` method associates the `NAME` scoped value with the current user. The `run()` method executes the provided code

block (a lambda expression in this case) within the scope of the bound value. The code within the `run()` method can access the scoped value using the `get()` method. In our case, the `doSomething()` method will access the value by calling the `get()` method:

```
NAME.get()
```

If we now want to replace the `JobScheduler` class with `ScopedValue`, we will do as follows:

```
private static final ScopedValue<JobContext> CONTEXT
                                                = ScopedValue.newInstance(); ❶

  public void schedule(Job job, String jobName, Priority priority) {
    JobContext context = new JobContext(jobName, priority); ❷
    ScopedValue.where(CONTEXT, context)
        .run(() -> runJob(job)); ❸
  }

  private void runJob(Job job) {
    job.execute(); ❹
  }

  public static JobContext getContext() { ❺
    return CONTEXT.get();
  }

  public static Object getJobMetadata(String key) {
    JobContext context = CONTEXT.get(); ❻
    if (context != null) {
      return context.getMetadataValue(key);
    }
    return null;
  }
}
```

Key aspects of this implementation:

❶ Creates a `ScopedValue` instance using the factory method

❷ Constructs the `context` object to be bound

❸ Binds the `context` and executes the job within that scope

❹ The job executes with access to the scoped `context`

❺ Provides static access to the current `context` (cleaner than the original)

❻ Safely retrieves metadata with `null` checking

Now that we understand how it works, let's take a more in-depth look.

While the previous examples demonstrate the use of `run()` to execute code within a scope, `ScopedValue` also provides a `call()` method when you need to return a value from the scoped execution. The difference is straightforward: `run()` executes a `Runnable` (returns void), while `call()` executes a `Callable` (returns a value).

Consider the following code snippets:

```
private static final ScopedValue<Double> DISCOUNT_RATE
                                        = ScopedValue.newInstance();

  public double calculatePrice(double basePrice) {
    // Using call() to return the calculated price from within the scope
    return ScopedValue.where(DISCOUNT_RATE, 0.20)  // 20% discount
        .call(() -> basePrice * (1 - DISCOUNT_RATE.get()));
  }

  void main() {
    PricingService service = new PricingService();
    double finalPrice = service.calculatePrice(100.0);
    System.out.println("Final price: $" + finalPrice);
    // Output: Final price: $80.00
  }
}
```

In this example, we use `call()` because we need to return the calculated price. This pattern is particularly useful for computations, transformations, or any operation where you need to retrieve a result while maintaining access to the scoped context.

Running ScopedValue

`ScopedValue` has a bounded lifetime. We must first set it, and then we can use it.

Consider the following example:

```
public static void main(String[] args) {

    ScopedValue<String> NAME = ScopedValue.newInstance();

    Runnable task = () -> {
        if (NAME.isBound()) {
            System.out.println("Name is bound: " + NAME.get());
        } else {
            System.out.println("Name is not bound");
        }
    };
```

```
    task.run();
}
```

If we run this code, it will print `Name is not bound` because we haven't set the value yet. To bind a value and execute code within that scope, we use the `where()` and `run()` methods:

```
    public static void main(String[] args) {
     ScopedValue<String> NAME = ScopedValue.newInstance();

     Runnable task = () -> {
         if (NAME.isBound()) {
             System.out.println("Name is bound: " + NAME.get());
         } else {
             System.out.println("Name is not bound");
         }
     };

     ScopedValue.where(NAME, "Bazlur") ❶
                .run(task); ❷
}
```

This approach:

❶ Binds the value `Bazlur` to the `NAME` scoped value

❷ Executes the task within the scope of that binding

This code will run in the context of the main thread, as the `main` method starts it. If we run the preceding code, we will see the following output:

```
Name is bound: Bazlur
```

Now, one might ask: after setting the `ScopedValue` using `where().run()`, can we execute the task separately and get the value?

Let's test this:

```
public static void main(String[] args) {
    ScopedValue<String> NAME = ScopedValue.newInstance();
    Runnable task = () -> {
        if (NAME.isBound()) {
            System.out.println("Name is bound: " + NAME.get());
        } else {
            System.out.println("Name is not bound");
        }
    };
    // Execute within scope
```

```
        ScopedValue.where(NAME, "Bazlur").run(task); ❶
        // Try to execute outside scope
        task.run(); ❷
    }
```

This demonstrates that:

❶ The first execution runs within the scoped value's bound context.

❷ The second execution runs outside that scope, so the value is no longer bound.

The output of this code will be:

```
Name is bound: Bazlur
Name is not bound
```

The `ScopedValue` only remains bound within the dynamic scope of the `run()` method call. Once that method completes, the binding is automatically removed, ensuring clean scope boundaries and preventing value leakage between unrelated code sections.

NOTE

The "scope" of a value defines where it exists and where it can be accessed. In Java, this often refers to *lexical scope* (defined by `{}` blocks) where variables are accessible only within their declared boundaries. However, `ScopedValue` operates on a *dynamic scope*, which is determined by the program's *execution flow*.

Dynamic scope means a value is accessible during the execution of specific methods and the methods they call, directly or indirectly. For example, if `a` calls `b`, and `b` calls `c`, the scope flows through `c` but ends when `c` finishes. `ScopedValue` binds a value to this execution flow, making it available only within the scope of the `run` method that sets it.

This temporary and precise scoping is what makes `ScopedValue` a cleaner and safer alternative to `ThreadLocal`, especially for passing contextual data.

We can ask another question: instead of using the main thread, can we run the task in another thread?

Consider the following code:

```
public static void main(String[] args) throws InterruptedException {
    ScopedValue<String> NAME = ScopedValue.newInstance();

    Runnable task = () -> {
```

```
            if (NAME.isBound()) {
                System.out.println("Name is bound: " + NAME.get());
            } else {
                System.out.println("Name is not bound");
            }
        };

        Thread thread = Thread.ofPlatform().unstarted(task); ❶
        ScopedValue.where(NAME, "Bazlur")
                   .run(thread::start); ❷

        thread.join();
    }
```

In this code:

❶ Creates an unstarted platform thread with our task

❷ Binds the scoped value and starts the thread within that scope

When we run this code, the output is:

```
Name is not bound
```

The reason we get this result is that scoped values are not automatically inherited by newly created threads. Although `thread::start` executes within the scope where `NAME` is bound, the actual task runs in a separate thread that doesn't inherit the scoped value binding. This behavior is the same for both platform and virtual threads.

Now, instead of creating the `ScopedValue` in the main thread, let's move it to the newly created thread:

```
    public static void main(String[] args) throws InterruptedException {
        ScopedValue<String> NAME = ScopedValue.newInstance();

        Runnable task = () -> {
            if (NAME.isBound()) {
                System.out.println("Name is bound: " + NAME.get());
            } else {
                System.out.println("Name is not bound");
            }
        };

        Thread thread = Thread.ofVirtual().start(() -> { ❶
            ScopedValue.where(NAME, "Bazlur")
                       .run(task); ❷
        });
```

```
        thread.join();
    }
```

Now the code works as expected:

❶ Creates and starts a virtual thread

❷ Binds the scoped value and runs the task within the new thread's context

The `ScopedValue` provides a fluent API; using it, we can bind multiple `ScopedValue` chains together.

Let's look at the following example:

```
public class MultiScopedExample {
    private static final ScopedValue<String> USER_ID
                                            = ScopedValue.newInstance();
    private static final ScopedValue<String> SESSION_ID
                                            = ScopedValue.newInstance();

    public static void main(String[] args) {
        ScopedValue.where(USER_ID, "user123") ❶
                .where(SESSION_ID, "session456") ❷
                .run(() -> performTask()); ❸
    }

    public static void performTask() {
        String userId = USER_ID.get();
        String sessionId = SESSION_ID.get();
        System.out.println("Performing task for user: " + userId +
                " in session: " + sessionId);
        logAction();
    }

    public static void logAction() {
        String userId = USER_ID.get(); ❹
        String sessionId = SESSION_ID.get();
        System.out.println("Logging action for user: " + userId +
                " in session: " + sessionId);
    }
}
```

This chaining approach:

❶ Binds the first scoped value

❷ Chains another binding

❸ Executes code with both values bound

❹ Allows both values to remain accessible throughout the call stack

`ScopedValue` has two other convenient methods, `orElse` and `orElseThrow`. These two methods are useful when we want to use a default value in case no value is available in the `ScopedValue` or if we want to throw an exception.

Consider the next example:

```
public class ScopedValueDefaultsExample {
  private static final ScopedValue<String> USER_NAME
      = ScopedValue.newInstance();

  public static void main(String[] args) {
    // Using orElse for default values
    String userNameUnbound = USER_NAME.orElse("Guest"); ❶
    System.out.println("No binding -> user name defaults to: "
        + userNameUnbound);
    // Using orElseThrow for validation
    try {
      USER_NAME.orElseThrow(() ->
          new IllegalStateException("No user name bound yet!")); ❷
    } catch (IllegalStateException e) {
      System.out.println("Caught exception: " + e.getMessage());
    }
    // Within a bound scope
    ScopedValue.where(USER_NAME, "Bazlur").run(() -> {
      String boundUserName = USER_NAME.orElse("Guest"); ❸
      System.out.println("Within binding -> user name is: " + boundUserName
      // This won't throw since the value is bound
      String validatedName = USER_NAME.orElseThrow(()
          -> new IllegalStateException("No user name bound yet!")); ❹
      System.out.println("Validated name: " + validatedName);
    });
  }
}
```

These methods:

❶ Provide safe access with a default value when unbound

❷ Allow for validation that throws a custom exception when unbound

❸ Return the bound value (ignoring the default)

❹ Can use the no-argument version when you're certain a value exists

This API design ensures that code can handle both bound and unbound states gracefully, making scoped values robust for various use cases.

Rebinding ScopedValue in nested scopes

One of the powerful features of `ScopedValue` is its ability to *rebind* within nested scopes. Rebinding allows you to assign a new value to the same `ScopedValue` for a limited duration, confined to a specific subscope. Once the subscope finishes, the original value is automatically restored, ensuring clean and predictable context management.

This rebinding feature is particularly useful in scenarios such as role-based access control, where a user's role can be temporarily switched during a specific operation. It also enables context-sensitive configurations, for example, allowing settings to be adjusted for a particular execution flow without affecting the broader context. Additionally, it can support task-specific overrides by providing temporary data for a specific task or action.

Now take a look at this example:

```
public class ScopedValueRebindingExample {
  private static final ScopedValue<String> USER_ROLE = ScopedValue.newInsta
  public static void main(String[] args) {
    // Bind initial value in outer scope
    ScopedValue.where(USER_ROLE, "Admin").run(() -> { ❶
      System.out.println("Outer scope: User role is " + USER_ROLE.get());
      performTask();
      // Rebind in nested scope
      ScopedValue.where(USER_ROLE, "Guest").run(() -> { ❷
        System.out.println("Inner scope: User role is " + USER_ROLE.get())
        performTask();
      }); ❸
      // Original value restored automatically
      System.out.println("Back to outer scope: User role is " + USER_ROLE.
      performTask();
    });
  }
  public static void performTask() {
    System.out.println("  Performing task as: " + USER_ROLE.get()); ❹
  }
}
```

Key aspects of this example:

❶ Establishes the initial binding of `Admin` in the outer scope.

❷ Creates a nested scope that temporarily rebinds the value to `Guest`.

❸ When this scope exits, the original `Admin` value is restored.

❹ The same method accesses different values based on the current scope.

If we run this code, we will get the following output:

```
Outer scope: User role is Admin
  Performing task as: Admin
Inner scope: User role is Guest
  Performing task as: Guest
Back to outer scope: User role is Admin
  Performing task as: Admin
```

ScopedValue and Structured Concurrency

`ScopedValue` is designed to work seamlessly with structured concurrency. As we have seen in the preceding example, `ScopedValue` isn't inherited by child threads; however, when used within a `StructuredTaskScope`, `ScopedValue` bindings are automatically inherited by all child threads created within that scope. This inheritance mechanism facilitates efficient data sharing between parent and child threads without explicitly passing the data as arguments. This is because structured concurrency has well-defined boundaries. When we exit the `StructuredTaskScope`, all the threads created in the scope are interrupted and then garbage collected; thus, there is no issue with memory leaks.

Let's look at an example:

```
import java.util.concurrent.StructuredTaskScope;

public class ScopedValueStructuredConcurrencyExample {
    private static final ScopedValue<String>
            USERNAME = ScopedValue.newInstance();

    public static void main(String[] args) {
        ScopedValue.where(USERNAME, "Bazlur").run(() -> {
            doSomething();
        });
    }

    public static void doSomething() {
        try (var scope = StructuredTaskScope.open()) {
            StructuredTaskScope.Subtask<String> task1 = scope.fork(()
                    -> USERNAME.get() + " from task 1");
            StructuredTaskScope.Subtask<String> task2 = scope.fork(()
                    -> USERNAME.get() + " from task 2");
            scope.join();
            String result1 = task1.get();
            String result2 = task2.get();
            System.out.println(result1);
            System.out.println(result2);
        } catch (InterruptedException e) {
```

```
                throw new RuntimeException(e);
            }
        }
    }
```

In this example, the `USERNAME` scoped value is bound to the string `Bazlur` in the main thread. When `doSomething()` is called, it creates a `StructuredTaskScope` and forks two child threads. These child threads inherit the `USERNAME` binding from the parent thread and can access it using `USERNAME.get()`.

Performance Considerations

`ScopedValue` generally shows better performance compared to `ThreadLocal`, especially when working with virtual threads. This performance advantage is the result of several factors. First, there is reduced overhead: `ThreadLocal` variables can introduce significant overhead when each virtual thread needs its own copy, which drives up memory consumption. In contrast, `ScopedValue` lets us share data among threads within a defined scope, minimizing memory usage and boosting performance. Second, `ScopedValue` is optimized for virtual threads and structured concurrency. It leverages the lightweight nature of virtual threads to provide efficient data sharing without the usual bottlenecks of traditional thread-local variables. For example, consider a web server that spins up thousands of virtual threads to handle concurrent requests. Each request may need contextual data like user authentication details. By using `ScopedValue` to share this data within each request's scope, we significantly reduce memory consumption and improve overall throughput compared to using `ThreadLocal`.

Usability and API Design

Beyond performance, `ScopedValue` brings several usability advantages that make it a better alternative to `ThreadLocal`. First, `ScopedValue` enforces immutability, which means once a value is associated with a `ScopedValue` within a scope, it cannot be modified. This immutability simplifies reasoning about the code and reduces the risk of errors caused by unintended modifications. In a concurrent environment, immutability is especially useful as it prevents race conditions and data inconsistencies that often arise when multiple threads attempt to modify shared variables.

Second, `ScopedValue` explicitly defines the lifecycle of shared data through its API design. The `run()` method clearly marks the scope where the value is accessible. This explicit lifecycle improves code readability and makes it easier to understand how data flows through the pro-

gram. Unlike the implicit behavior of `ThreadLocal`, this design ensures a clearer boundary for where the value is valid.

Third, the API of `ScopedValue` is more concise and intuitive. The `where()` and `run()` methods provide a structured way to set and access shared data within a specific scope. Compared to the often verbose and less straightforward methods of `ThreadLocal`, `ScopedValue`'s approach feels more natural and easier to work with.

Additionally, `ScopedValue` acts as a capability object, which allows precise control over who can access the shared data. By declaring the `ScopedValue` object with access modifiers like `private`, it becomes possible to restrict access to authorized components only, adding a layer of security and encapsulation to the design.

Finally, `ScopedValue` handles `null` values more explicitly than `ThreadLocal`. With `ThreadLocal`, calling `get()` will return `null` whether the value was explicitly set to `null` or never set at all, which can lead to subtle bugs. In contrast, `ScopedValue` makes a clear distinction. If the value is unbound, calling `get()` throws a `NoSuchElementException`, ensuring any oversight in setting the value is caught early, making the code more robust and predictable.

Migrating to Scoped Values

Scoped values are likely to be useful and preferable in many scenarios where thread-local variables are used today. Beyond serving as hidden method parameters, scoped values can be particularly useful in several areas.

First, sometimes we want to detect recursion, perhaps because a framework is not re-entrant or because recursion must be limited in some way. A scoped value provides a way to detect and handle the recursion.

You can set it up as usual, with `ScopedValue.run()`, and then deep in the call stack, call `ScopedValue.isBound()` to check if it has a binding for the current thread. More elaborately, the scoped value can model a recursion counter by being repeatedly rebounded.

Consider a document-processing system that handles nested templates. Without a recursion limit, a maliciously crafted template with circular reference could crash your application. Let's look at the following code:

```
public class TemplateProcessor {
    private static final ScopedValue<Integer> RECURSION_DEPTH
                                        = ScopedValue.newInstance(); ❶
```

```
    private static final int MAX_NESTING_LEVEL = 50;

    public String processTemplate(String template) {
        if (!RECURSION_DEPTH.isBound()) { ❷
            return ScopedValue.where(RECURSION_DEPTH, 0) ❸
                    .call(() -> processTemplateInternal(template));
        } else {
            return processTemplateInternal(template);
        }
    }
}
```

❶ Creates a `ScopedValue` to track recursion depth across method calls

❷ Uses `isBound()` deep in the call stack to check if it has a binding for the current thread

❸ Sets it up as usual with `ScopedValue.run()` (note: `where()` and `call()` are from the modern API)

The main processing logic handles the template parsing and recursion management. This method is called within the scoped context and has access to the current recursion depth:

```
private String processTemplateInternal(String template) {
    int currentDepth = RECURSION_DEPTH.get(); ❹

    if (currentDepth >= MAX_NESTING_LEVEL) { ❺
      throw new TemplateProcessingException(
          "Template nesting too deep: " + currentDepth + " levels");
    }

    StringBuilder result = new StringBuilder();

    // Simplified template processing logic
    int includeStart = template.indexOf("{{include:");
    if (includeStart >= 0) {
      int includeEnd = template.indexOf("}}", includeStart);
      String includePath = template.substring(includeStart + 10, includeEnd

      String nestedContent = ScopedValue
          .where(RECURSION_DEPTH, currentDepth + 1) ❻
          .call(() -> processTemplateInternal(loadTemplate(includePath)));

      result.append(template, 0, includeStart);
      result.append(nestedContent);
      result.append(template.substring(includeEnd + 2));
    } else {
      result.append(template);
    }
```

```
        return result.toString(); ❼
    }
```

❹ Retrieves the current recursion depth within the scope.

❺ Implements a safety check to prevent excessive nesting in non-reentrant frameworks.

❻ The scoped value models a recursion counter by being repeatedly rebounded with incremented values.

❼ When this method returns, the previous depth value is automatically restored.

To complete our implementation, we need supporting methods for template loading and error handling. The `loadTemplate()` method simulates reading templates from a filesystem, including some that create deep nesting for testing:

```
private String loadTemplate(String path) {
    return switch (path) {
      case "header.tpl" -> "<!-- HEADER START -->";
      case "footer.tpl" -> "<!-- FOOTER END -->";
      default -> {
        if (path.startsWith("level")) {
          int level = Integer.parseInt(path.replaceAll("[^0-9]", ""));
          if (level < 10) {
            yield "Level " + level
                + " {{include:level" + (level + 1) + ".tpl}}";
          }
        }
        yield "<!-- Template content from " + path + " -->";
      }
    };
}
```

Let's see how this all works together. The following `main` method demonstrates various usage scenarios, from simple templates to deeply nested ones that trigger our recursion protection:

```
void main() {
        TemplateProcessor processor = new TemplateProcessor();

        // Example 1: Simple template without nesting
        String simpleTemplate = "Hello, this is a simple template!";
        System.out.println("Simple template result:");
        System.out.println(processor.processTemplate(simpleTemplate));
        System.out.println();

        // Example 2: Template with nested includes
        String nestedTemplate = "Header: {{include:header.tpl}} " +
```

```
                                        "Content goes here {{include:footer.tpl}}";
            System.out.println("Nested template result:");
            System.out.println(processor.processTemplate(nestedTemplate));
            System.out.println();

            // Example 3: Demonstrate recursion depth tracking
            String deeplyNested = "Level 0 {{include:level1.tpl}}";
            System.out.println("Processing deeply nested template...");
            try {
                String result = processor.processTemplate(deeplyNested);
                System.out.println("Result: " + result);
            } catch (TemplateProcessingException e) {
                System.err.println("Error: " + e.getMessage());
            }
        }
```

When we run the preceding code, we get the following result:

```
Simple template result:
Hello, this is a simple template!

Nested template result:
Header: <!-- HEADER START --> Content goes here <!-- FOOTER END -->

Processing deeply nested template...
Result: Level 0 Level 1 Level 2 Level 3 Level 4 Level 5 Level 6 Level 7 Lev
Level 9 <!-- Template content from level10.tpl -->
```

Second, detecting recursion can also be useful in the case of flattened transactions: any transaction started while a transaction is in progress becomes part of the outermost transaction.

Consider the following simplified example:

```
public class FlattenedTransactionExample {
  private static final ScopedValue<Transaction> CURRENT_TRANSACTION =
      ScopedValue.newInstance();

  public static void main(String[] args) {
    performBusinessOperation();
  }

  private static void performBusinessOperation() {
    // Start outer transaction
    Transaction outerTx = new Transaction("OUTER_TX");
    ScopedValue.where(CURRENT_TRANSACTION, outerTx).run(() -> { ❶
      System.out.println("Starting: " + outerTx.name());
      // Nested operation that might start its own transaction
      performNestedOperation();
      System.out.println("Committing: " + outerTx.name());
```

```
        });
    }

    private static void performNestedOperation() {
        if (CURRENT_TRANSACTION.isBound()) { ❷
            // Join existing transaction
            Transaction currentTx = CURRENT_TRANSACTION.get();
            System.out.println("  Joining existing transaction: " + currentTx.nam
            performDatabaseOperation();
        } else {
            // Start new transaction if none exists
            Transaction newTx = new Transaction("NESTED_TX");
            ScopedValue.where(CURRENT_TRANSACTION, newTx).run(() -> {❸
                System.out.println("  Starting new transaction: " + newTx.name());
                performDatabaseOperation();
            });
        }
    }

    private static void performDatabaseOperation() {
        Transaction tx = CURRENT_TRANSACTION.get();
        System.out.println("    Executing in transaction: " + tx.name()); ❹
    }

    record Transaction(String name) {
    }
}
```

This pattern:

❶ Establishes the outermost transaction

❷ Detects whether a transaction is already active

❸ Only creates a new transaction if none exists

❹ Means all operations participate in the same transaction context

The preceding code uses `ScopedValue` to manage nested transactions. It starts an outer transaction and binds it to a `ScopedValue`. When a nested transaction is attempted, the code checks if a transaction is already bound, effectively joining the outer transaction. This ensures all operations, even nested ones, are part of the same overarching transaction, which simplifies management and maintains data consistency.

If we run this code, we will get the following output:

```
Starting: OUTER_TX
  Joining existing transaction: OUTER_TX
    Executing in transaction: OUTER_TX
Committing: OUTER_TX
```

Another example occurs in graphics, where there is often a drawing context to be shared between parts of the program. `ScopedValue` s, because of their automatic cleanup and reentrancy, are better suited to this than thread-local variables.

Consider a typical graphics application where drawing operations, such as colors, line width, and transformations, need to share state. Traditional approaches using thread-local variables require careful manual cleanup and are prone to context leakage. `ScopedValue` would provide a cleaner, safer alternative.

Let's examine how `ScopedValue` s can manage drawing context in a component hierarchy:

```
import java.awt.Color;

public class SimpleGraphicsExample {


    // Drawing context as ScopedValues
    private static final ScopedValue<Color> DRAW_COLOR
                               = ScopedValue.newInstance(); ❶
    private static final ScopedValue<Integer> LINE_WIDTH
                               = ScopedValue.newInstance();

    // Simulated drawing methods
    static void drawLine(String from, String to) {
        Color color = DRAW_COLOR.isBound() ? DRAW_COLOR.get() : Color.BLACK
        int width = LINE_WIDTH.isBound() ? LINE_WIDTH.get() : 1;

        System.out.printf("Drawing line from %s to %s [Color: %s, Width: %d
            from, to, color.toString(), width);
    }

    static void drawRectangle(String name) {
        Color color = DRAW_COLOR.isBound() ? DRAW_COLOR.get() : Color.BLACK
        int width = LINE_WIDTH.isBound() ? LINE_WIDTH.get() : 1;

        System.out.printf("Drawing rectangle '%s' [Color: %s, Width: %d]\n
            name, color.toString(), width);
    }
```

❶ Drawing properties are declared as `ScopedValue` s, providing thread-safe context management.

❷ The `isBound()` check ensures we have a fallback when no context is established.

Components in a UI hierarchy often need their own drawing styles while inheriting from their parents:

```
// Component that draws itself and its children
    static void drawButton(String label) {
        System.out.println("\n--- Drawing Button: " + label + " ---");

        // Button uses blue color with thick border
        ScopedValue.where(DRAW_COLOR, Color.BLUE) ❸
            .where(LINE_WIDTH, 3)
            .run(() -> {
                drawRectangle("button-background");

                // Text inside button uses different color
                ScopedValue.where(DRAW_COLOR, Color.WHITE) ❹
                    .where(LINE_WIDTH, 1)
                    .run(() -> {
                        System.out.println("Drawing text: " + label);
                    });

                // Border automatically uses blue again
                drawRectangle("button-border"); ❺
            });
    }
```

❸ The button creates its own drawing context with blue color and 3-pixel line width.

❹ Text rendering temporarily overrides the color to white, demonstrating safe nesting.

❺ After the text scope ends, the button's blue context is automatically restored.

Containers can establish a drawing context that affects all their children:

```
// Panel that contains multiple components
    static void drawPanel() {
        System.out.println("\n--- Drawing Panel ---");

        // Panel uses gray theme
        ScopedValue.where(DRAW_COLOR, Color.GRAY)
            .where(LINE_WIDTH, 2)
            .run(() -> {
                drawRectangle("panel-background");

                // Draw child components - each with their own style
                drawButton("OK"); ❻
                drawButton("Cancel");

                // Back to panel's gray automatically
                drawLine("divider-start", "divider-end"); ❼
            });
    }
```

❻ Each button maintains its own drawing context, unaffected by the panel's gray theme.

❼ After drawing children, the panel's context remains intact.

Now let's look at our `main` method to complete the example:

```
void main() {
        System.out.println("=== Graphics Context Example ===\n");

        // Set default drawing context
        ScopedValue.where(DRAW_COLOR, Color.BLACK) ❽
            .where(LINE_WIDTH, 1)
            .run(() -> {
                // Draw with default black color
                drawLine("A", "B");

                // Draw a panel (which has its own colors)
                drawPanel();

                // Automatically back to black after panel
                System.out.println("\n--- Back to main context ---");
                drawLine("C", "D"); ❾
            });

        // Outside the scope - no context available
        System.out.println("\n--- Outside any context ---");
        drawLine("E", "F");  // Will use defaults ❿
    }
}
```

❽ Establishes the default drawing context for the entire application.

❾ After the panel completes, the main context is automatically restored without manual intervention.

❿ Operations outside any context fall back to sensible defaults.

Running this example produces the following output, clearly showing context changes:

```
=== Graphics Context Example ===
Drawing line from A to B [Color: java.awt.Color[r=0,g=0,b=0], Width: 1]
--- Drawing Panel ---
Drawing rectangle 'panel-background' [Color: java.awt.Color[r=128,g=128,b=
Width: 2]
--- Drawing Button: OK ---
Drawing rectangle 'button-background' [Color: java.awt.Color[r=0,g=0,b=255
Width: 3]
Drawing text: OK
Drawing rectangle 'button-border' [Color: java.awt.Color[r=0,g=0,b=255],
Width: 3]
```

```
--- Drawing Button: Cancel ---
Drawing rectangle 'button-background' [Color: java.awt.Color[r=0,g=0,b=255
Width: 3]
Drawing text: Cancel
Drawing rectangle 'button-border' [Color: java.awt.Color[r=0,g=0,b=255],
Width: 3]
Drawing line from divider-start to divider-end [Color: java.awt.Color
[r=128,g=128,b=128], Width: 2]
--- Back to main context ---
Drawing line from C to D [Color: java.awt.Color[r=0,g=0,b=0], Width: 1]
--- Outside any context ---
Drawing line from E to F [Color: java.awt.Color[r=0,g=0,b=0], Width: 1]
```

Now that we understand the benefits of `ScopedValue`, let's consider a few important factors before migrating. First, the API is still in preview, so it's a good idea to familiarize yourself with it and explore its capabilities until it becomes available in the JDK without requiring preview flags. This way, you can seamlessly adopt it once it is officially released. Additionally, migrating to a newer JDK should be a key consideration, as `ScopedValue` is only available in the latest JDK. Evaluate any constraints you might face before upgrading to ensure a smooth transition.

When using `ScopedValue`, it's important to ensure that the data we intend to share is immutable, as `ScopedValue` enforces immutability. `ScopedValue` might not be the right fit if our use case requires mutable data. Defining the scope of shared data explicitly with the `where()` and `run()` methods is equally crucial. This ensures the data remains accessible within the intended context, avoiding unintended side effects.

We should also be mindful of the differences in `null` value handling between `ScopedValue` and `ThreadLocal`. `ScopedValue` throws a `NoSuchElementException` when accessing unbound values, whereas `ThreadLocal` returns `null`. This explicit error handling can help catch issues early but requires careful attention during migration.

Finally, it is essential to thoroughly test your application after adopting `ScopedValue` to ensure that concurrency and data consistency are preserved and to address any unforeseen issues that may arise.

In Closing

`ScopedValue` offers a cleaner, safer, and more efficient alternative to `ThreadLocal` for sharing context across a call chain. By being immutable and bounded to a well-defined dynamic scope, it addresses many of `ThreadLocal`'s shortcomings: unconstrained mutability, unbounded lifetime, and memory overhead. Its lightweight nature aligns particularly

well with virtual threads and structured concurrency, helping reduce resource usage when spinning up large numbers of concurrent tasks. The explicit syntax of `where()` and `run()` makes it clear when and where a given value is in effect, improving code readability and maintainability.

As of JDK 25, `ScopedValue` has graduated from preview status and is now a stable API, ready for production use. This stabilization marks it as the recommended solution for context propagation in modern Java applications. We can confidently migrate existing `ThreadLocal`-based code to use scoped values without concern about API changes.